Code559

Front-End Developer Course

MODULE 3

Responsive Layout

One page that works on a phone, a tablet and a laptop.

Free lesson: 559solutions.com/code559/module-03-responsive-layout/

In this packet

1. **The lesson**
2. **Project: A responsive Interests page**
3. **Finishing work in Git**
4. **Troubleshooting: five common problems**
5. **Self-assessment**
6. **Answers**

Powered by 559 Solutions

About this module

What this module covers

You learn the box model, mobile-first media queries, CSS Grid and responsive images, and how to test a page at different widths. It's essential because more visitors will see your site on a phone than on a laptop, and one HTML file with one stylesheet can serve them all.

Tips for the learner

Some prompts ask you to explain code to yourself, out loud. Others ask you to write something down. Each habit does a different job.

Recite to understand. When a prompt says "out loud," explain the code in your own words, as if you were teaching it. Saying it aloud makes a deeper impression than reading silently. It helps you picture how the pieces of code fit together. And it shows you right away which line you cannot explain yet. That line is the one to review.

Write it down to check yourself. When a prompt says "write down," it is usually a prediction, a choice, or a guess about a bug. A written answer can't change after you see the result, so the comparison is fair and you learn from the difference. Writing also keeps your attention on the lesson and helps you pick out the main points.

Keep one notebook for this course. Use it for your predictions, key terms, steps, mistakes, and questions to come back to. Date each entry and write the module number, so you can find it later. Before you start a new module, read your last few entries and recite one or two of them out loud.

The four badges

Every core concept moves through the same four steps, called IEIA. Each step is marked with its badge.

- **Introduce** (a lightbulb: a new idea) - You meet the concept: its name, and why it matters to you, in a sentence or two.

- **Explain** (a speech bubble: the idea in words) - How it works, one idea at a time, in plain words.

- **Illustrate** (an eye: see it) - A small example you can look at: a code snippet, a real screenshot or a diagram. Its caption, which starts with this badge, tells you what to notice.

- **Apply** (a keyboard: your turn) - A short task to do right now, before you read on. Often you type or predict something; sometimes you write it down, sketch it, or explain code to yourself out loud.

Rights and credits

© 2026 559 Solutions LLC. This packet is licensed to the purchaser for personal use. Do not share, resell or post it.

Portions adapted from Visual Studio Code documentation (CC BY 3.0 US), GitHub Docs (CC BY 4.0), and GitHub Skills (MIT). Screenshots used with permission from Microsoft.

Contents

About this module 2

Part 1: The lesson 4

- Why this matters 4
- Before you start 4
- Core concepts 5
  - 1. What is the box model? 5
  - 2. What does `box-sizing: border-box` change? 6
  - 3. What does the viewport meta tag do? 6
  - 4. How does a mobile-first media query work? 7
  - 5. What makes navigation responsive? 8
  - 6. How does CSS Grid make a card layout? 9
  - 7. How do responsive images work? 11
  - 8. How do you test at different widths? 13
- Exercise 13
- Common misconception 15
- Summary 15
- Check your understanding 16
- Before next module 16
- Go further 16
- What to keep 16

Part 2: Project: A responsive Interests page 18

Part 3: Finishing work in Git 28

Part 4: Troubleshooting: five common problems 30

Part 5: Self-assessment 32

Part 6: Answers 33

About 559 Solutions 36

© 2026 559 Solutions LLC

PART 1

The lesson

One page that works on a phone, a tablet and a laptop.

Time: about 2 hours (estimate)

In this module you will learn:

- The box model: content, padding, border and margin
- What `box-sizing: border-box` changes, and why most stylesheets start with it
- What the viewport meta tag does, and what happens on a phone without it
- Mobile-first media queries with `min-width`, and why the order of your rules matters
- A responsive navigation bar that stacks on a phone and sits in a row on a laptop
- CSS Grid: `display: grid`, `repeat(auto-fit, minmax(220px, 1fr))`, `gap` and `grid-column: span 2`
- Responsive images: `srcset` with `w` descriptors, `sizes`, `width` and `height` to stop layout shift, `loading="lazy"`, and `figure` with `figcaption`
- How to test a page at different widths with the DevTools device toolbar

---

Why this matters

More people will open your site on a phone than on a laptop. If the page only looks right on the screen you built it on, most visitors see a broken version. Responsive layout means one HTML file and one stylesheet that adapt to the width they are given.

You do not need a separate phone site, and you do not need JavaScript. The tools in this module are plain CSS and a few HTML attributes. Once you know them, every page you build for the rest of the course starts on a phone and grows.

This module also brings in CSS Grid. In Module 2 you used Flexbox to put things in a row. Grid puts things in rows **and** columns at the same time, which is exactly what a page of cards needs.

Before you start

1. Module 2 is done: you can link a page to an external stylesheet, and you have used Flexbox to put a navigation bar in a row.
2. Open your course folder (for example `Documents\code559`). Next to `my-site`, create a new folder called `practice-03`. This is the throwaway folder for the Exercise. Nothing in it is published.
3. Open Edge or Chrome and press **F12** once to check that DevTools opens. Press F12 again to close it.

© 2026 559 Solutions LLC

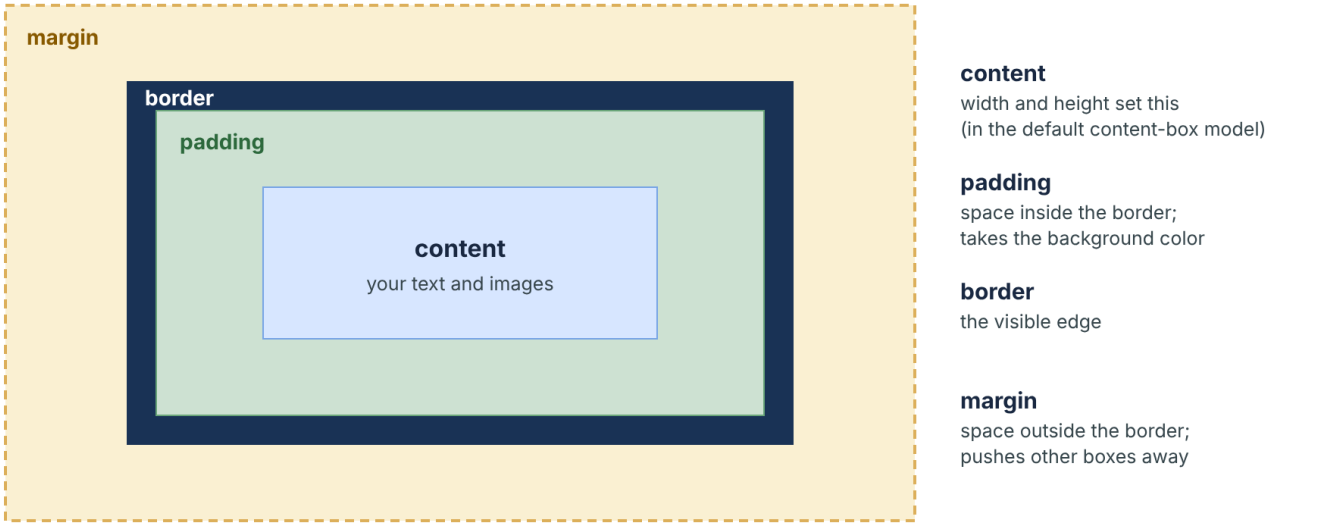
> **Checkpoint.** You have an empty `practice-03` folder next to `my-site`, and F12 opens DevTools in your browser.

Core concepts

1. What is the box model?

Every element on a page is a rectangle. CSS calls that rectangle a box, and every box has four layers.

From the inside out: the **content** is your text or image. **Padding** is space inside the border; it takes the element's background color. The **border** is the visible edge. **Margin** is space outside the border that pushes other boxes away. MDN's *The box model* page says that in the default model, "if you set `width` and `height` property values on a box, these values define the `width` and `height` of the *content box*. Any padding and borders are then added." So a box with `width: 300px`, `padding: 20px` and a `5px` border is really 350 pixels wide on the screen.



Margin is outside the border, padding is inside it, and `width` sets only the content in the default model.

Open any page you built in Edge or Chrome, press F12, click an element in the Elements panel and find the Computed tab. DevTools draws that element's box with all four layers and their sizes.

© 2026 559 Solutions LLC

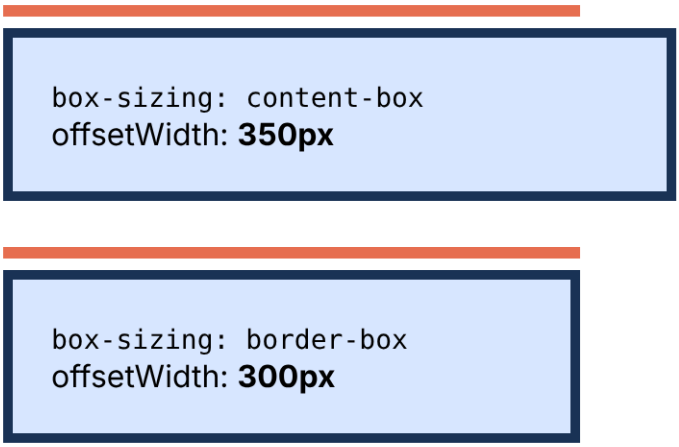
2. What does `box-sizing: border-box` change?

`box-sizing` decides what `width` measures.

With `box-sizing: border-box`, MDN says "any width is the width of the visible box on the page." The padding and border fit **inside** the width you set, so 300 means 300. That makes layouts much easier to reason about, which is why many stylesheets begin with this rule for every element:

```
*, *::before, *::after {
  box-sizing: border-box;
}
```

Two boxes with the same CSS (`width: 300px; padding: 20px; border: 5px solid`). The only difference is `box-sizing`. A small script on the page wrote each box's real width into it.



The orange bars are 300 px wide. The default box is 350 px; the border-box one is exactly 300 px. The numbers are `offsetWidth`, measured by the page in Chromium. (real: rendered in Chromium)

Predict: with `border-box`, what happens to the content area if you raise the padding from 20px to 40px? (It shrinks by 40 px; the outside stays 300 px.)

3. What does the viewport meta tag do?

You have been typing this line in every `<head>` since Module 1:

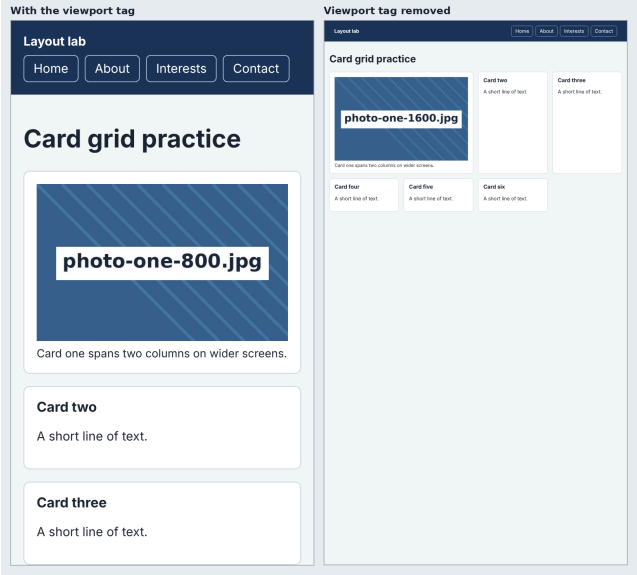
```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

MDN's page on `<meta name="viewport">` explains that some phones lay a page out in a "virtual viewport" that is wider than the screen, for example 980 px, "and then it will be shrunk down to fit." When that happens, MDN goes on, media queries meant for narrow screens "will never be used." The

© 2026 559 Solutions LLC

viewport tag tells the phone to use its real width. `initial-scale=1` asks it to start at 100% zoom.

You will see this for yourself in step 8 of the Exercise. Here is the result in advance.



Same page, same 390 px phone, in Chromium's phone emulation. With the tag removed, the page measured `innerWidth` 980 and laid out 4 grid columns, then shrank everything to fit. (real: rendered in Chromium)

Look at the `<head>` of your Home page now. Is the viewport line there, spelled exactly? If not, add it.

4. How does a mobile-first media query work?

A media query is a block of CSS that only applies when a condition is true, such as "the viewport is at least 700 px wide."

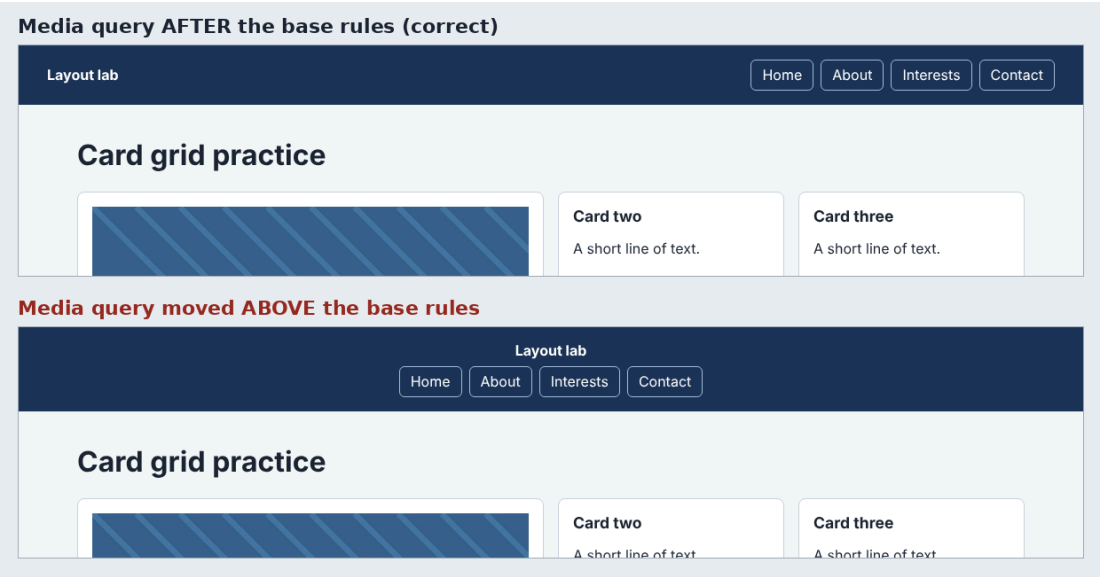
Mobile first means you write the phone layout as your normal rules, then add `@media (min-width: …)` blocks that change only what is different on wider screens. MDN's *Using media queries* page notes that range features like `width` take `min-` and `max-` prefixes, and that these comparisons are inclusive, so `min-width: 700px` is true at exactly 700.

The order matters. A rule inside a media query has the same specificity as the same rule outside it. When two rules tie, **the later one wins** (you met this in Module 2's cascade). So the media query must come **after** the base rules it changes. If you put it first, the base rule comes later and always wins.

```
/* Base rules: the phone layout */
.site-header {
  display: flex;
  flex-direction: column;
}
```


© 2026 559 Solutions LLC

```
/* Wider screens: added on top of the base rules */
@media (min-width: 700px) {
  .site-header {
    flex-direction: row;
    justify-content: space-between;
  }
}
```



Both renders are 1200 px wide, so the media query is true in both. Moved above the base rules, it loses: Chromium measured `flex-direction: column` instead of `row`. (real: rendered in Chromium)

Predict: with the CSS above, which `flex-direction` does the header get on a 500 px screen? (`column`: the media query is false below 700 px, so only the base rule applies.)

5. What makes navigation responsive?

Navigation is usually the first thing to break on a phone. Four links in a row fit on a laptop and overflow on a phone.

The simplest responsive navigation needs no JavaScript. On a phone, the header stacks (logo above links) and the links use `flex-wrap: wrap` so they flow onto a second line if they must. At 700 px and up, the media query from Concept 4 puts the logo and the links in one row. Give each link some padding so it is easy to tap with a finger. Menus that open and close with a button need JavaScript, which comes in Module 9.

Compare the top of the 390 px and 1200 px screenshots in Concept 6 below. Same HTML, same stylesheet.

© 2026 559 Solutions LLC

Open your `practice-02/styles.css` from Module 2. Add `flex-wrap: wrap` to the `nav` rule and `padding: 0.4rem 0.75rem` to the `nav a` rule. Save and press **F5**. If the links ever run out of room on a narrow window, they now move onto a second line instead of running off the edge.

6. How does CSS Grid make a card layout?

Grid lays out children in columns and rows. You turn it on with `display: grid` on the parent.

This one line does most of the work:

```
.card-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 1rem;
}
```

Read it from the inside out. `1fr` is, in MDN's words in *Basic concepts of grid layout*, "a fraction of the available space in the grid container." `minmax(220px, 1fr)` means "each column is at least 220 px, and shares any extra space equally." `repeat(auto-fit, …)` means "make as many of those columns as will fit." MDN's `repeat()` page adds that `auto-fit` collapses any empty columns, so a short row of cards stretches to fill the width. `gap` puts space between the cells; MDN notes it is taken out before the `fr` space is shared.

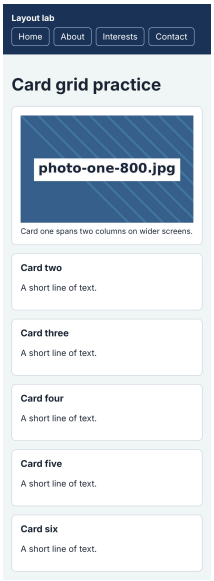
There is no media query in that rule. The number of columns changes with the width by itself. Here is what Chromium measured on the Exercise page (`getComputedStyle(grid).gridTemplateColumns`):

Viewport width	Columns measured	Each column
390 px	1	358px
800 px	3	245.328px
1200 px	4	255px

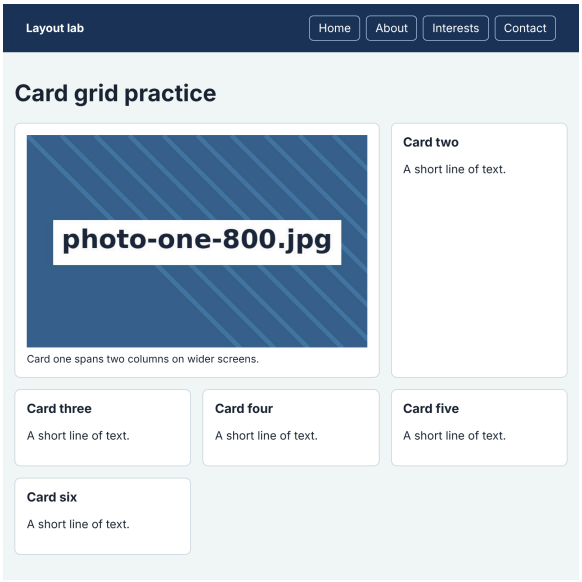
One card can be wider than the others with `grid-column: span 2`. Put that inside the `min-width` media query. When it was applied at all widths, Chromium measured `220px 122px` at 390 px: the span forced a second, squeezed column onto a phone.

© 2026 559 Solutions LLC

The minimum in `minmax()` always holds: if the space is narrower than one minimum-width column, Grid still makes that column at the minimum width, and it sticks out past the edge, so the page scrolls sideways. And if only one column fits but a card has `span 2`, Grid adds a column anyway, and that extra column comes out narrower than the first one, as in the `220px 122px` measurement above.

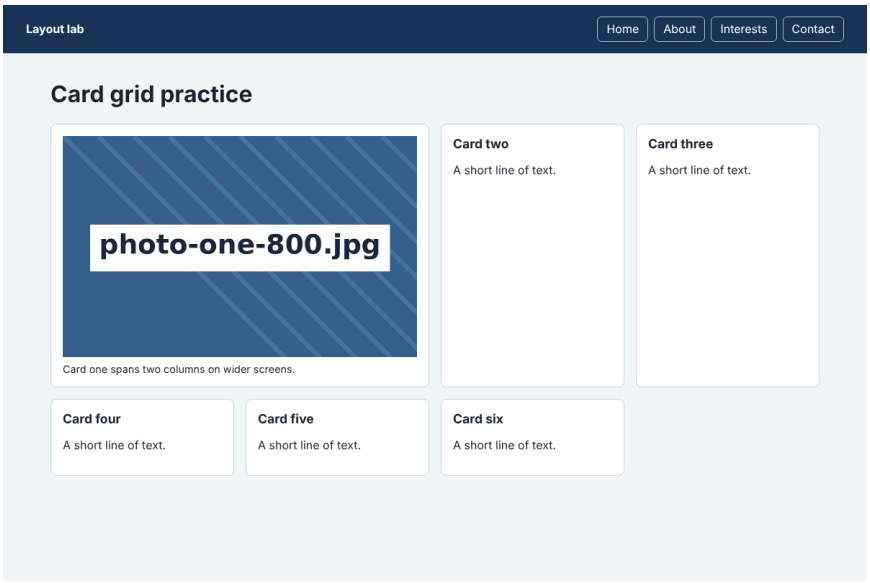


390 px (a phone): one column, header stacked. (real: rendered in Chromium at 390 px)



800 px (a tablet): three columns, and the featured card spans two of them. (real: rendered in Chromium at 800 px)

© 2026 559 Solutions LLC



1200 px (a laptop): four columns. `main` has `max-width: 1100px`, so the cards stop growing. Notice that cards two and three are as tall as card one: items in the same grid row stretch to the row's height by default. (real: rendered in Chromium at 1200 px)

Predict: how many columns at 1000 px? Write your guess down. You will check it in the Exercise.

7. How do responsive images work?

A photo big enough for a laptop is a waste of data on a phone. HTML lets you offer several sizes and lets the browser choose.

Four attributes and one pair of elements do the job:

- **`srcset`** lists the files and their real widths with a `w` descriptor. MDN's *Using responsive images in HTML* points out that `480w` "uses the `w` unit, not `px` as you might expect." The number must match the file's real pixel width.
- **`sizes`** tells the browser how wide the image will be shown, before the CSS has loaded. `(min-width: 700px) 50vw, 100vw` means "about half the viewport on wider screens, the full width otherwise." The browser uses the first condition that is true.
- **`width` and `height`** are the image's real pixel size, with no unit. MDN's `<img>` page says they let the browser work out the aspect ratio "prior to the image being loaded" and reserve the space, which stops the text below from jumping. Keep `img { max-width: 100%; height: auto; }` in your CSS so the image still shrinks.
- **`loading="lazy"`** waits to download an image until it is near the screen. MDN warns that lazy images especially need `width` and `height`, and that lazy loading only happens when JavaScript is enabled.
- **`<figure>` and `<figcaption>`** wrap an image and its caption so they belong together.

```
<figure>
  <img src="images/photo-one-800.jpg"
```

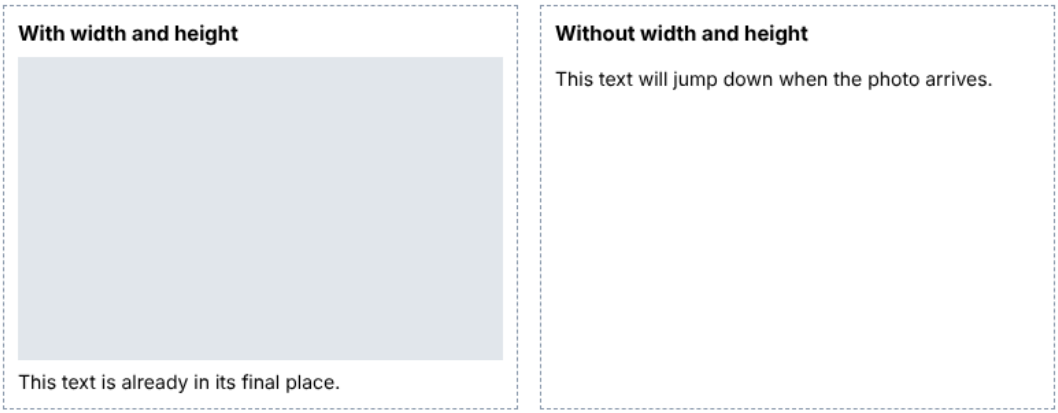

© 2026 559 Solutions LLC

```
        srcset="images/photo-one-400.jpg 400w,
                images/photo-one-800.jpg 800w,
                images/photo-one-1600.jpg 1600w"
        sizes="(min-width: 700px) 50vw, 100vw"
        width="1600" height="1000"
        loading="lazy"
        alt="Describe what the photo shows">
  <figcaption>A caption the reader can see.</figcaption>
</figure>
```

The browser also counts pixel density. A phone screen often has two or three device pixels per CSS pixel, so it picks a bigger file. The demo images have their own file name painted on them, so you can see which one loaded. Chromium chose (`img.currentSrc`, measured):

Viewport	1 pixel per CSS px	2 pixels per CSS px
390 px	photo-one-400.jpg	photo-one-800.jpg
800 px	photo-one-400.jpg	photo-one-800.jpg
1200 px	photo-one-800.jpg	photo-one-1600.jpg

To see the reserved space, the page below asks for two photos that never arrive. The left one has `width` and `height`; the right one does not.



While the photos were still loading, Chromium measured the left image at 221 px tall and the right one at 0 px. When the right photo arrives, its text jumps down. (real: rendered in Chromium at 800 px)

Pick the picture on your `practice-02` page. Find its real pixel size (right-click the file in File Explorer, **Properties → Details**). In its `<img>` tag, set `width` to the real width (replace `320` if it is there) and add `height` with the real height. Then add `img { max-width: 100%; height: auto; }` to

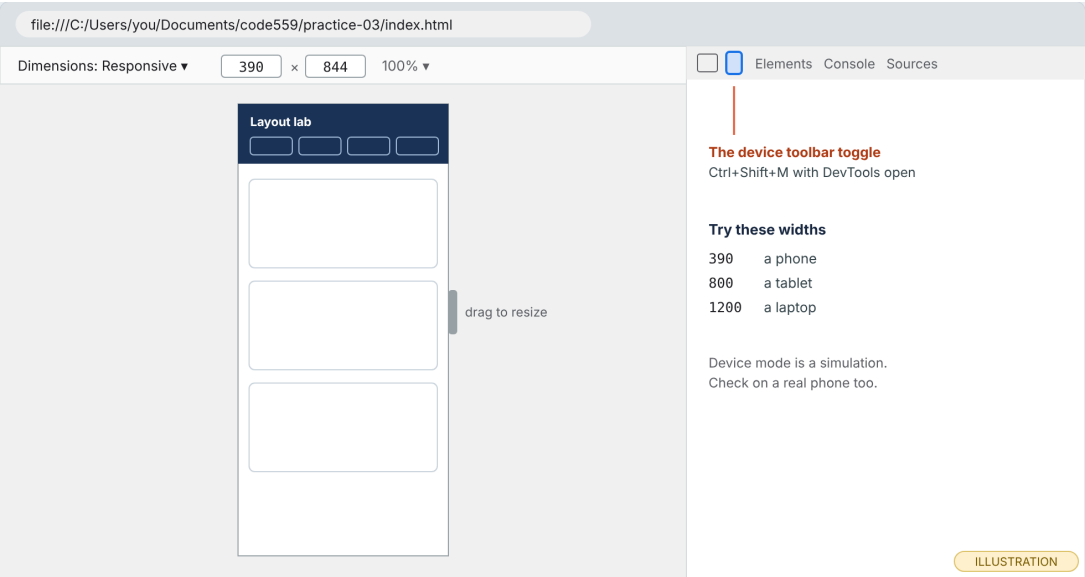
© 2026 559 Solutions LLC

`practice-02/styles.css` so the picture still fits the window. Save and press **F5**. The picture may now show larger, but never wider than the page.

8. How do you test at different widths?

You do not need three devices to test three widths. DevTools can resize the page for you.

Open DevTools (F12), then press **Ctrl+Shift+M** (Chrome DevTools' shortcut for *Toggle Device Mode*; Edge uses the same DevTools). A toolbar appears above the page. Leave the dimensions on Responsive and type a width: 390, then 800, then 1200. You can also drag the handle on the side of the page. Chrome's documentation calls device mode "a first-order approximation" of a phone, so check on a real phone when you can.



The device toolbar sits above the page; the toggle is next to the Elements tab. Labels in your browser may differ slightly.

Open your Home page in Edge or Chrome, open device mode (F12, then Ctrl+Shift+M), and try 390, 800 and 1200. Write down one thing that looks wrong at 390.

Exercise

Your hands, about 50 minutes (estimate). Work in `practice-03`. Type it, do not paste it. The `srcset` and the grid line especially should go through your fingers once.

1. **Make the files.** In `practice-03`, create `index.html`, `styles.css` and a folder called `images`. Put three copies of any one photo in `images`, resized to 400, 800 and 1600 px wide, and name them `photo-one-400.jpg`, `photo-one-800.jpg` and `photo-one-1600.jpg`. (The Windows

© 2026 559 Solutions LLC

Photos app can resize; any image editor works.)

2. **Write the HTML.** The normal head, with the viewport line and a link to `styles.css`. In the body: a `<header class="site-header">` with a logo link (a plain text link with your site's name, such as `<a href="index.html">My site</a>`; no image needed) and a `<nav>` holding a `<ul class="nav-list">` of four links; then a `<main>` with an `<h1>` and a `<ul class="card-grid">` of six `<li class="card">` items. Give the first card `class="card featured"` and put the `<figure>` from Concept 7 in it. In its `<img>`, change `height="1000"` to the real height of your `photo-one-1600.jpg` (right-click the file, **Properties → Details**); `1000` is right only for a photo that is 1600 × 1000. The other five get an `<h2>` and a `<p>`.

3. **Write the base CSS, phone first.** The `box-sizing` rule from Concept 2. `img { max-width: 100%; height: auto; display: block; }`. The stacked `.site-header` from Concept 4. `.nav-list` with `display: flex; flex-wrap: wrap; gap: 0.5rem; list-style: none; padding: 0;`. `main { max-width: 1100px; margin: 0 auto; padding: 1rem; }`. The `.card-grid` rule from Concept 6 (add `list-style: none; padding: 0;`). Give `.card` some padding, a border and a white background. Add `body { margin: 0; }` and `figure { margin: 0; }`: browsers give the page and every figure a margin by default. Without the `body` rule your column widths will not match the table in Concept 6; without the `figure` rule the photos do not fill their cards as they do in the pictures.

4. **Add the media query last.** At the bottom of `styles.css`, one `@media (min-width: 700px)` block that turns the header into a row and gives `.featured` `grid-column: span 2`.

5. **Look at three widths.** Open `index.html` in Edge or Chrome (double-click it in File Explorer, or drag it onto a browser window). After each save, press **F5** to reload. Press **F12** (or **Ctrl+Shift+J**) to open DevTools, then click the **Console** tab. With DevTools open, press **Ctrl+Shift+M** to turn on device mode, and check 390, 800 and 1200. You should see 1, 3 and 4 columns. Now check your prediction from Concept 6 at 1000.

6. **Check the image.** Click in the Console. Type this line exactly as shown. You will learn how it works in Module 9; for now, read the answer it gives.

   ```
   document.querySelector('.featured img').currentSrc
   ```

   Which file did your browser choose? Change the width, press **F5**, then type the line again. Device mode's Responsive setting counts 2 device pixels per CSS pixel (step 8a says why), so compare your answers with the "2 pixels per CSS px" column of the table in Concept 7.

7. **Watch the columns change.** Click in the Console. Type this line exactly as shown. You will learn how it works in Module 9; for now, read the answer it gives.

   ```
   getComputedStyle(document.querySelector('.card-grid')).gridTemplateColumns
   ```


© 2026 559 Solutions LLC

The answer is a list of column widths. Count them.

8. **Break it on purpose (two ways).**

   - **a. Remove the viewport line.** Delete the `<meta name="viewport" …>` line, save, and press **F5** with device mode on at 390. The page is shrunk and the text is tiny. You do not need to pick a phone from the Dimensions menu: Responsive already acts like a phone by default, because the Chrome DevTools source code (the file `DeviceModeModel.ts`) sets its default device type to Mobile and its default pixel ratio to 2. Click in the Console. Type this line exactly as shown. You will learn how it works in Module 9; for now, read the answer it gives.

     ```
     innerWidth
     ```

     The answer no longer matches the phone's 390. Chromium reported 980 when this lesson tested it the same way (a phone at 390 px, pixel ratio 2). Put the line back.

   - **b. Move the media query to the top.** Cut the `@media` block and paste it above the `.site-header` rule. Save, press **F5** and look at 1200 px. The header is stacked again. Write one sentence: why did the base rule win? Then move the block back to the bottom.

Common misconception

> *"Responsive design means writing a separate stylesheet for phones."*

What would you say? Hint: look at how many media queries the Exercise page used, and how many times the number of grid columns changed.

Summary

Every element is a box of content, padding, border and margin, and `box-sizing: border-box` makes `width` mean the visible width. The viewport meta tag makes a phone lay the page out at its real width, so your media queries can fire. Mobile first means phone rules first, then `min-width` media queries below them, because the later of two equal rules wins. Responsive navigation stacks on a phone and becomes a row in a media query. `repeat(auto-fit, minmax(220px, 1fr))` makes as many columns as fit, `gap` spaces them, and `grid-column: span 2` widens one card. `srcset` with `w` descriptors and `sizes` let the browser pick an image file, `width` and `height` reserve its space, `loading="lazy"` delays it, and `figure` with `figcaption` keeps it with its caption. The DevTools device toolbar lets you test any width without another device.

© 2026 559 Solutions LLC

Check your understanding

1. A box has `width: 200px; padding: 10px; border: 2px solid;` and no `box-sizing` rule. How wide is it on screen? What if you add `box-sizing: border-box`?
2. Your `@media (min-width: 700px)` rule works at 1200 px on your laptop but never fires on your phone. Name the first thing you would check.
3. **What happens if this changes?** In `minmax(220px, 1fr)`, you change 220px to 400px. What happens to the number of columns at 800 px and at 390 px?

Before next module

- [ ] `practice-03` shows 1, 3 and 4 columns at 390, 800 and 1200 px
- [ ] You have seen both breaks in step 8 and put both back
- [ ] Your `practice-03` page has the viewport line and the `box-sizing` rule
- [ ] The `@media` block in `practice-03/styles.css` is below the rules it changes
- [ ] If you did this module's packet project in my-site, saved it with the routine: git status → git add . → git commit -m "..." → git push → check the live page. (Your practice folder is not saved to GitHub.)

Go further

- MDN: *The box model*; *Using media queries*; *Basic concepts of grid layout*; *repeat()*; *Using responsive images in HTML*; the `<img>` element page; `<meta name="viewport">`
- Chrome for Developers: *Simulate mobile devices with device mode*; *Keyboard shortcuts* (DevTools)
- freeCodeCamp: the responsive web design and CSS Grid lessons in the current curriculum (search "CSS Grid")

What to keep

Keep `practice-03` with the media query back at the bottom, and a note of which image file your browser chose at each width. Then answer for yourself: **which step would be hardest to repeat without this packet, and why?**

Code559 is a 559 Solutions project. Portions adapted from Visual Studio Code documentation (CC BY 3.0 US), GitHub Docs (CC BY 4.0), and GitHub Skills (MIT). Screenshots used with permission from Microsoft.

Sources checked 2026-10-09: MDN "The box model"; MDN "Using media queries"; MDN "Basic concepts of grid layout"; MDN "repeat()"; MDN "Using responsive images in HTML"; MDN "`<img>`: The Image Embed element"; MDN "`<meta name="viewport">`"; Chrome for Developers "Simulate mobile devices with device mode" and "Keyboard shortcuts". Checked 2026-10-11: Chrome DevTools source code, `front_end/models/emulation/DeviceModeModel.ts` (GitHub, ChromeDevTools/devtools-frontend: Responsive defaults to Mobile with pixel ratio 2); HTML Standard "Rendering" (default margins of `body` and `figure`). Measurements made with Chromium 141 (headless, via Playwright).

© 2026 559 Solutions LLC

PART 2

Project: A responsive Interests page

Your hands, about 90 minutes (estimate). In the Exercise you built a responsive practice page in `practice-03`. Now you add a third page to your real site. You build an **Interests** page in `my-site` as a card grid: one card per interest, each with a photo, a caption and a sentence about why you like it. On a phone the cards sit in one column. On a laptop they spread into a grid, and your favorite spans two columns. Every photo comes in three sizes, and the browser picks the one it needs. You also make the shared header mobile first, so all three pages work on a phone. You use only HTML and CSS from Modules 1 to 3. There is no JavaScript.

This project starts from the Module 2 packet's site: Home, About and one shared `styles.css`. If your site does not have that yet, finish the Module 2 packet project first (Part 2), then come back.

The brief

When you finish, `my-site` holds `index.html`, `about.html`, the new `interests.html`, `image-one.jpg`, `styles.css`, and a new folder `images` with your interest photos. All three pages link the same `styles.css`. The live address is `https://your-username.github.io/my-site/`.

The nav. Every page's nav has four links, in this order: **Home**, **About**, **Interests**, **Contact**. The new link is `<a href="interests.html">Interests</a>`. The header is still the same, letter for letter, on all three pages.

The Interests page. It has the same `<head>`, header and footer as the About page, with the title `Interests - Your Name`. Its `<main>` has one `<section>` with an `<h2>` that says **My interests**, a short opening paragraph with `class="lead"`, and a `<ul class="card-grid">` with one `<li class="card">` for each interest. Use four to six interests; the sample has five. Each card holds, in this order:

- an `<h3>` with the name of the interest,
- a `<figure>` with the `<img>` and a `<figcaption>` that says what the photo shows,
- a `<p>` with one sentence about why you like it.

Your favorite interest is the first card, and it has `class="card featured"`.

The photos. Each interest has one photo saved in three widths: 400, 800 and 1200 pixels. They live in `my-site\images` and are named `interest-one-400.jpg`, `interest-one-800.jpg`, `interest-one-1200.jpg`, then `interest-two-…` and so on. Each `<img>` has a `srcset` with all three files and their `w` descriptors, a `sizes` attribute, `width` and `height` set to the real size of the 1200 file, `loading="lazy"`, and real `alt` text.

The stylesheet. `styles.css` gets three changes, and you keep the four groups from Module 2:

- At the very top, the `box-sizing: border-box` rule from Concept 2, and in the site-wide basics, an `img` rule with `max-width: 100%` and `height: auto`.

© 2026 559 Solutions LLC

- The header and nav rules become mobile first. On a phone, the site name sits above the nav, and the nav links can wrap onto a second line.
- Two new groups at the end: **Interests page** (the grid, the cards, the figure and the caption), and last of all **Wider screens**, one `@media (min-width: 700px)` block that turns the header into a row and makes `.featured` span two columns.

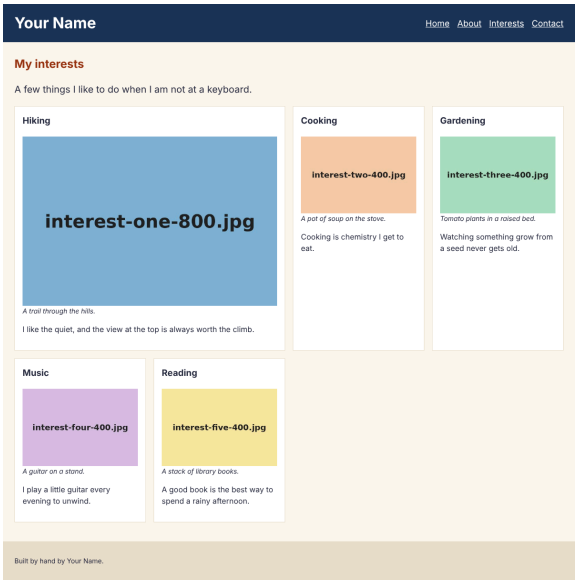
A visitor on a phone sees the site name above the four links, then the cards in one column. A visitor on a laptop sees the header as one row and the cards in a grid of three or four columns. Nothing scrolls sideways, and the text does not jump down while the photos load. All the words and photos are yours. The class names, file names, the `700px` breakpoint and the `minmax(220px, 1fr)` grid line must match.

The pictures in this packet use simple colored placeholder images with their own file name painted on them, so you can see which file the browser chose. You use your own photos.

© 2026 559 Solutions LLC



The Interests page on a phone. The header stacks, and the cards sit in one column, each photo as wide as its card. At pixel ratio 1 the browser chose the 400 files. Only the top of the page is shown. (real: rendered in Chromium at 390 px)



The same page on a laptop: four columns of 276 px each, measured by Chromium. The featured card spans two of them, so the browser chose a bigger file for it. (real: rendered in Chromium at 1200 px)

The new card colors were checked: the dark text `#2b2d42` on the white cards is 13.5:1 (computed with the WCAG contrast formula). The card border uses the footer's beige `#e9dcc9`. It is decoration, so it has no contrast minimum.

The steps

Type every line. Do not paste. The one exception is the header and footer, which you copy from `about.html`, as you did in Module 1.

© 2026 559 Solutions LLC

1. **Open `my-site`.** In VS Code, **File → Open Folder...** and choose `my-site` in your course folder, `C:\Users\you\Documents\code559`. Open a terminal with **Terminal → New Terminal** and check that the prompt ends in `my-site>`.

2. **Plan on paper.** Write down four to six interests. Circle your favorite: it becomes the featured card. For each one, write a short caption (what the photo shows) and one sentence about why you like it.

3. **Prepare the photos.** Pick one photo for each interest. In the VS Code Explorer, create a folder called `images` inside `my-site`. Save each photo three times into it, resized to 400, 800 and 1200 pixels wide, with the names from the brief: `interest-one-400.jpg`, `interest-one-800.jpg`, `interest-one-1200.jpg`, and so on. The Windows Photos app can resize; any image editor works. Then write down the real size of each 1200 file (right-click it in File Explorer, **Properties → Details**). A photo resized to 1200 wide from a 3 by 2 photo is 1200 × 800, but phone photos are often 4 by 3 (1200 × 900) or tall. Write down what Windows tells you, not what you expect.

4. **Predict.** Write down how many columns of cards you expect at 390, 800 and 1200 px, and which file (400, 800 or 1200) you expect the browser to choose for a normal card at 390 px. You will check both in steps 12 and 13.

5. **Add the box rules.** Open `styles.css`. At the very top, above the site-wide basics, type the rule from Concept 2 with its own comment:

   ```
   /* Every box: width includes padding and border */
   *, *::before, *::after {
     box-sizing: border-box;
   }
   ```

   Then, in the site-wide basics group, after the `a` rule, add an `img` rule:

   ```
   img {
     max-width: 100%;
     height: auto;
     display: block;
   }
   ```

   `max-width: 100%` keeps a photo from ever being wider than its box. `height: auto` keeps its shape when it shrinks. `display: block` removes the small gap that an inline image leaves under itself. Save, open `about.html` in Edge or Chrome (double-click it in File Explorer, or drag it onto a browser window), and press **F5** after each save. The About page should look the same as before: your picture is still 320 px wide.

© 2026 559 Solutions LLC

6. **Make the header mobile first.** Look at your Home page in device mode first. Press **F12**, then **Ctrl+Shift+M**, and type 390 as the width. In the Module 2 header, the site name and the links share one row, so on a phone the name may break onto two lines and the links get squeezed. Now change the header and nav rules so the phone layout is the base:

```
/* Header and nav */
header {
  background-color: #1d3557;
  color: white;
  padding: 16px 24px;
  display: flex;
  flex-direction: column;
  align-items: flex-start;
  gap: 8px;
}

nav {
  display: flex;
  flex-wrap: wrap;
  gap: 8px 16px;
}
```

   The header lost `justify-content: space-between` and `align-items: center`; they come back in step 10, for wide screens only. `flex-direction: column` stacks the site name above the nav, and `align-items: flex-start` keeps both at the left edge. In the `nav` rule, `flex-wrap: wrap` lets the links move onto a second line if they run out of room. A `gap` with two values sets the space between rows first (8px), then between columns (16px). It replaces Module 2's `gap: 24px`, so four links fit in one row on a phone. In your `nav a` rule, add `padding: 4px 0;` so each link is a little taller and easier to tap. Save and press F5. At 390 px the name now sits above the links.

7. **Add the Interests link.** In `index.html`, add the new link between About and Contact:

```
<nav>
  <a href="index.html">Home</a>
  <a href="about.html">About</a>
  <a href="interests.html">Interests</a>
  <a href="index.html#contact">Contact</a>
</nav>
```

   Make the same change in `about.html`. Then read the two headers line by line: they must still match.

© 2026 559 Solutions LLC

8. **Make the Interests page.** In the Explorer, create `interests.html`. Copy all of `about.html` into it. Change the title to `Interests - Your Name`, and replace everything between `<main>` and `</main>` with this skeleton:

```
<main>
  <section>
    <h2>My interests</h2>
    <p class="lead">One sentence that introduces your interests.</p>
    <ul class="card-grid">
      <li class="card featured">
        <h3>Your favorite interest</h3>
        <figure>
          <img src="images/interest-one-800.jpg"
               srcset="images/interest-one-400.jpg 400w,
                       images/interest-one-800.jpg 800w,
                       images/interest-one-1200.jpg 1200w"
               sizes="(min-width: 700px) 66vw, 100vw"
               width="1200" height="800"
               loading="lazy"
               alt="Describe what the photo shows">
          <figcaption>A short caption.</figcaption>
        </figure>
        <p>One sentence about why you like it.</p>
      </li>
      <!-- Continue alone: one li class="card" for each other interest, with the
           same parts. Use sizes="(min-width: 700px) 33vw, 100vw" for these. -->
    </ul>
  </section>
</main>
```

Set `height` to the real height of your `interest-one-1200.jpg` from step 3. The `800` in the skeleton is right only for a 1200 × 800 photo. Do the same for every card. The card headings are `<h3>`, because they sit inside a section whose heading is an `<h2>`.

Read the `sizes` values to yourself. On a screen 700 px or wider, the featured card is about two thirds of the window and a normal card about one third; below 700 px every card is the full width. These are estimates for the browser, made before the CSS loads. They do not have to be exact. They only help it pick a file.

9. **Style the cards.** At the end of `styles.css`, add the Interests page group:

```
/* Interests page */
.card-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 16px;
```


© 2026 559 Solutions LLC

```
  list-style: none;
  padding: 0;
}

/* Continue alone: a .card rule with a white background, a 1px border in a
   light color, and 16px padding; a .card h3 rule with margin-top: 0;
   figure with margin: 0; and figcaption with a smaller, italic font. */
```

`list-style: none` and `padding: 0` remove the bullets and the indent that every `<ul>` has by default. `figure { margin: 0; }` matters: browsers give every figure a margin, and without this rule your photos do not fill their cards.

10. **Add the wider-screens block last.** At the very end of `styles.css`, below everything else:

```
/* Wider screens: 700px and up */
@media (min-width: 700px) {
  header {
    flex-direction: row;
    justify-content: space-between;
    align-items: center;
  }

  /* Continue alone: a .featured rule that spans two grid columns. */
}
```

Inside the block you write only what is different on a wide screen. The header keeps its color, padding and `display: flex` from the base rule.

11. **Run.** Save everything. Open `interests.html` in the browser and press F5. Turn on device mode (F12, then Ctrl+Shift+M) and try 390, 800 and 1200. Then click **Home**, **About** and **Interests** at each width. The header should look the same on all three pages: stacked at 390, one row at 800 and 1200.

12. **Count the columns.** Press **F12** (or **Ctrl+Shift+J**) to open DevTools, then click the **Console** tab. Type this line exactly as shown. You will learn how it works in Module 9; for now, read the answer it gives.

```
getComputedStyle(document.querySelector('.card-grid')).gridTemplateColumns
```

Compare with your prediction from step 4. On the sample page, Chromium measured one column (`342px`) at 390, three (`240px` each) at 800, and four (`276px` each) at 1200. Your numbers match if your `main` padding and grid gap match the sample; the column count is what matters.

© 2026 559 Solutions LLC

13. **Check which photo loaded.** Still in the Console, type this line exactly as shown. You will learn how it works in Module 9; for now, read the answer it gives.

    ```
    document.querySelector('.featured img').currentSrc
    ```

    The answer ends in the file name the browser chose. Device mode's Responsive setting counts 2 device pixels per CSS pixel, so it chooses bigger files than a plain laptop window. On the sample page, Chromium chose:

Viewport	Featured card, 1 pixel per CSS px	Featured card, 2 pixels per CSS px	Normal card, 1 pixel	Normal card, 2 pixels
390 px	400	800	400	800
800 px	800	1200	400	800
1200 px	800	1200	400	800

    Change the width, press **F5**, and ask again. Press F5 each time: a browser that already has a bigger file may keep showing it when you narrow the window. To ask about a normal card, change `.featured img` to `.card:nth-child(2) img`.

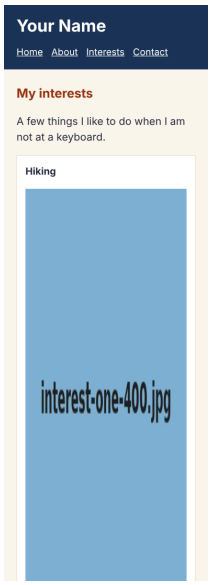
14. **Modify.** Change `minmax(220px, 1fr)` to `minmax(300px, 1fr)`. Save, press F5, and count the columns at 800 and 1200 again. Fewer, wider columns fit. Put back `220px`.

15. **Break it on purpose: the span outside the media query.** Cut the `.featured` rule out of the media query and paste it just above the `/* Wider screens */` comment. Save, and look at 390 px. Predict first, then look.

    The cards no longer sit in one wide column. Chromium measured `220px 129.172px` on the sample page: the featured card asked for two columns, so Grid made a second, narrow column, and every other card squeezed into it. The exact width of that second column depends on your card text; what matters is that there is a second, narrow column. This is the same thing you saw in the lesson's Concept 6. A span belongs in the wide-screen block. Move the rule back into the media query, save, and refresh.

16. **Break it on purpose: no `height: auto`.** In the `img` rule, delete `height: auto;`. Save and look at 390 px.

© 2026 559 Solutions LLC



Without `height: auto`, the photo is 308 px wide but 800 px tall, because the browser used the `height="800"` from the HTML. (real: rendered in Chromium at 390 px)

The photo is now 800 px tall and squashed. `max-width: 100%` shrank its width to fit the card, but nothing told the height to follow, so the browser used the `height` attribute as a fixed height. With `height: auto`, the browser still uses `width` and `height` to reserve the right *shape*, and it scales the height with the width. Put the line back.

17. **Debug: a photo that breaks at one width only.** In the featured card's `srcset`, change `images/interest-one-800.jpg 800w` to `images/intrest-one-800.jpg 800w` (drop the first `e`). Save. Turn device mode off, make the window narrow, and press F5: the photo may look fine. Now turn device mode on at 390 and press F5. The featured photo is broken, and the Console shows a red error for `intrest-one-800.jpg`. On the sample page at 390, Chromium loaded the 400 file at pixel ratio 1 and the missing 800 file at pixel ratio 2. A mistake in one `srcset` entry shows only on screens that choose that entry. That is why you test every width. Fix the name, save, and refresh.

18. **Test the site.** Tick each box.

    - [ ] At 390 px, all three pages show the site name above the four links, and nothing scrolls sideways
    - [ ] At 800 and 1200 px, the header is one row on all three pages
    - [ ] The Interests page shows 1, 3 and 4 columns at 390, 800 and 1200 px
    - [ ] The featured card spans two columns at 800 and 1200 px, and not at 390 px
    - [ ] Every photo loads at 390, 800 and 1200 px in device mode, with no red errors in the Console
    - [ ] Every photo has a caption and real `alt` text
    - [ ] All twelve nav clicks work: four links on three pages

© 2026 559 Solutions LLC

19. **Put it online.** Go to Part 3, *Finishing work in Git*, and publish your Interests page with the save routine. Then run the test from step 18 again on the live site, and on your own phone if you have one.

© 2026 559 Solutions LLC

PART 3

Finishing work in Git

Your Interests page works on your computer. Now you put it online with the same save routine: `git status`, `git add .`, `git commit`, `git push`, then check the live page. What is new this time is a new folder full of image files, and checking the live site at more than one width.

Items marked **NEW** are new in this part. The others repeat what you did in Modules 1 and 2.

1. **Check the terminal.** In VS Code, with `my-site` open, look at the terminal prompt. It must end in `my-site>`. If it does not, use **File → Open Folder...**, choose `my-site`, and open a new terminal.

2. **See what changed (NEW: a new folder).** Type:

   ```
   git status
   ```

   Under **Changes not staged for commit** you should see `modified:` for `about.html`, `index.html` and `styles.css`. Under **Untracked files** you should see `images/` and `interests.html`. Git lists a new folder as one line, `images/`, with a slash at the end. It does not list the files inside until you stage them.

3. **Stage the changes.** Type:

   ```
   git add .
   ```

   It prints nothing when it works. Run `git status` again. Now every photo is listed on its own line under **Changes to be committed**, as `new file: images/interest-one-400.jpg` and so on, next to `new file: interests.html` and the three `modified:` files. Count the photo lines: three per interest. If one is missing, check its name in the Explorer before you commit.

4. **Commit.** Type:

   ```
   git commit -m "Add a responsive Interests page with responsive images"
   ```

   The first line of the summary starts with `[main`, a seven-character commit id and your message. The next line says how many files changed: with five interests, `19 files changed` (15 photos, the new page, and three changed files). Then comes one `create mode 100644` line for each new file. Your commit id and numbers will differ. The new page, its photos, the new nav link and the stylesheet changes belong together, so they go into one commit.

5. **Push.** Type:

   ```
   git push
   ```


© 2026 559 Solutions LLC

Photos are bigger than text files, so this push may take a little longer. The last line ends in `main -> main`.

6. **Check that Git is done.** Type `git status`. It should say `nothing to commit, working tree clean`. Then type `git log --oneline`. Your new commit is on top, with your Module 2 commit under it.

7. **Check the live page (NEW: checking the photos online).** Wait a few minutes (GitHub Docs: "It can take up to 10 minutes for changes to your site to publish"), then open `https://your-username.github.io/my-site/interests.html` and press F5. Every card should show its photo. A broken photo online that works on your computer is almost always a capital letter: `Images` or `Interest-One-400.jpg` works on Windows, but the web server treats it as a different name (Module 1, *Troubleshooting*). Press F12, open the **Console** tab, and look for a red `404 (Not Found)` error with the file name.

8. **Test the live site at three widths (NEW).** With the live page open, turn on device mode (F12, then Ctrl+Shift+M) and run the test from Project step 18 again at 390, 800 and 1200 px. If you have a phone, open your live address on it too. Chrome's documentation calls device mode "a first-order approximation" of a phone, and the real phone is the final check.

> **Checkpoint.** `git status` says `nothing to commit, working tree clean`. `git log --oneline` shows your new commit on top. Your live Interests page shows every photo at 390, 800 and 1200 px, and the Console has no red errors.

© 2026 559 Solutions LLC

PART 4

Troubleshooting: five common problems

Each problem has the symptom you see, why it happens, and the fix. After each fix, save, press F5, and run the test from Project step 18 again.

1. The page is tiny and zoomed out on a phone

Symptom. In device mode at 390 px, or on your real phone, the Interests page looks like a shrunk copy of the laptop page: tiny text, the header in one row, and the cards in several columns.

Why. The page has no viewport meta tag, or it is misspelled. Without it, the phone lays the page out in a wider "virtual viewport" and shrinks it to fit (Concept 3). Your `min-width: 700px` query is then true on a phone. On the sample page without the tag, Chromium's phone emulation at 390 px reported `innerWidth` of about 980 and made four grid columns. The usual cause in this project is a new page made from scratch instead of copied from `about.html`.

Fix. Make sure the `<head>` of `interests.html` has this line, spelled exactly:

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

Check the other two pages too. Save and press F5 in device mode.

2. The cards stay in one column at every width

Symptom. At 1200 px the cards are still one on top of the other, each as wide as the page.

Why. The grid is not on the parent of the cards. Grid lays out the *children* of the element that has `display: grid`. If `display: grid` is in the `.card` rule, each card becomes a grid of its own, and the `<ul>` around them is a plain list. On the sample page with that mistake, every card measured 1152 px wide at 1200 px. A class name that does not match (`class="cards-grid"` in the HTML and `.card-grid` in the CSS) has the same result, with no error message.

Fix. Put `display: grid` and the `grid-template-columns` line in the `.card-grid` rule, and check that the `<ul>` in your HTML has exactly `class="card-grid"`. In the Console, the line from Project step 12 should give a list of pixel widths, such as `276px 276px 276px 276px`. If it gives the word `none`, the `<ul>` is not a grid: that is what Chromium returns when `display: grid` and the columns line are both in the `.card` rule. If it gives back your `repeat(…)` text, the columns line stayed on `.card-grid` but `display: grid` did not.

3. A photo is stretched, or the text jumps while photos load

Symptom. A photo is squashed into a tall, narrow shape. Or, on a slow connection, the caption and sentence sit right under the heading and then jump down when the photo arrives.

© 2026 559 Solutions LLC

Why. These are the two halves of one job. The `width` and `height` attributes let the browser reserve space of the right shape before the photo loads (Concept 7). If they are missing, the browser reserves nothing, and the text jumps. If they are there but the CSS has no `height: auto`, the browser shrinks the width to fit the card but keeps the `height` attribute as a fixed height, so the photo stretches (Project step 16). If `height` does not match the real photo, the shape is wrong too.

Fix. Every `<img>` needs `width` and `height` set to the real size of its 1200 file, as Windows shows it in **Properties → Details**. Your `img` rule needs both `max-width: 100%` and `height: auto`.

4. The browser picks a different file than you expected

Symptom. You expected `interest-two-400.jpg` at 390 px, but the Console line from Project step 13 says `interest-two-800.jpg`. Or a narrow window still shows a 1200 file.

Why. The browser chooses from `srcset` by device pixels, not CSS pixels. A screen with 2 device pixels per CSS pixel needs twice as many pixels to look sharp, so at 390 px it wants about 780 and picks the 800 file. Device mode's Responsive setting uses a pixel ratio of 2, and many phones use 2 or 3. That is the browser doing its job. Two other causes: a browser that already loaded a bigger file may keep using it when you make the window narrower, and a `w` number that does not match the file's real width misleads the browser.

Fix. Compare your answer with the 2-pixels-per-CSS-px column for your card type in the table in Project step 13. Press F5 after every change of width before you ask again. Check that each `w` number matches the real width of its file: `400w` for the 400 file, and so on.

5. The header stays stacked on a wide screen

Symptom. At 1200 px, the site name still sits above the links on every page, even though your media query says `flex-direction: row`.

Why. The `@media` block comes before the `header` rule in `styles.css`. A rule inside a media query has the same specificity as the same rule outside it, so the later one wins (Concept 4). On the sample page with the block moved above the header rules, Chromium measured `flex-direction: column` at 1200 px. The featured card still spanned two columns, because no later rule disagrees about `grid-column`. So a media query in the wrong place can break one thing and not another, which makes it easy to miss.

Fix. Move the whole `@media (min-width: 700px)` block to the very end of `styles.css`, below every rule it changes. Keep it last whenever you add new rules later.

© 2026 559 Solutions LLC

PART 5

Self-assessment

Rate yourself on each line: 1 Beginning, 2 Developing, 3 Proficient, 4 Ready to advance. Aim for at least **3 (Proficient)** on every line before you start Module 4. If a line is a 1 or a 2, reread that Core Concept and redo its Exercise or Project step.

Skill	1 Beginning	2 Developing	3 Proficient	4 Ready to advance
I can name the four layers of the box model and work out a box's real width with and without box-sizing: border-box	☐	☐	☐	☐
I can explain what the viewport meta tag does and spot a page that is missing it	☐	☐	☐	☐
I can write phone rules first and put a min-width media query after the rules it changes	☐	☐	☐	☐
I can make a header stack on a phone and become a row on a wider screen, with nav links that wrap	☐	☐	☐	☐
I can build a card grid with repeat(auto-fit, minmax(220px, 1fr)) and gap, and make one card span two columns only on wide screens	☐	☐	☐	☐
I can write an img with srcset, sizes, width, height, loading="lazy" and alt, inside a figure with a figcaption	☐	☐	☐	☐
I can explain why the browser chose a given srcset file, using the width and the pixel ratio	☐	☐	☐	☐
I can test a page at 390, 800 and 1200 px with the DevTools device toolbar and read the results in the Console	☐	☐	☐	☐

Three questions to answer in writing

1. In Project step 15, the featured card's span moved outside the media query and the phone layout broke. Explain why, in your own words, and say why the same rule is fine inside the media query.
2. Pick one mistake you made in this module. What did the page or the Console show, and how did you find the cause?
3. You add a sixth interest to the page. List everything you must make, add or check, in order, from the photo files to the live site.

© 2026 559 Solutions LLC

PART 6

Answers

Try each question on your own first, in writing. Then compare. The section in brackets is where the lesson or the project explains it.

Check your understanding

1. **The 200 px box.** Without a `box-sizing` rule, `width` sets only the content, so the box is 200 + 10 + 10 (padding) + 2 + 2 (border) = **224 px** wide on screen. With `box-sizing: border-box`, the padding and border fit inside the width, so it is **200 px**, and the content area shrinks to 176 px. (Core concepts 1 and 2)
2. **The query never fires on the phone.** Check the viewport meta tag first: `<meta name="viewport" content="width=device-width, initial-scale=1">` in the `<head>`, spelled exactly. Without it, the phone lays the page out at a wide virtual width, close to 980 px, and shrinks it to fit. Your `min-width: 700px` query is then *true* on the phone, so the phone gets the laptop layout. (Core concept 3; Exercise step 8a; *Troubleshooting*, problem 1)
3. **`minmax(400px, 1fr)`.** Fewer columns fit. On the Exercise page at 800 px, the grid has 768 px of room, and two 400 px columns plus the gap need 816, so there is only **one column** (Chromium measured `768px`). At 390 px there is only 358 px of room, but the minimum in `minmax()` always holds: the one column is **400 px** wide, sticks out past the edge, and the page scrolls sideways (Chromium measured a page 416 px wide in a 390 px window). (Core concept 6)

Common misconception

"Responsive design means writing a separate stylesheet for phones." The Exercise page used one HTML file, one stylesheet and **one** media query, and the number of grid columns still changed three times: 1, 3 and 4 at 390, 800 and 1200 px. Most of that came from `repeat(auto-fit, minmax(220px, 1fr))`, which has no media query at all. Responsive design is one set of base rules for the phone, plus small changes for wider screens. Your Interests page works the same way: three pages, one `styles.css`, one `@media` block. (Core concepts 4 and 6; Project steps 10 and 12)

Exercise checks

- **Step 5.** At 390, 800 and 1200 px you see 1, 3 and 4 columns. At 1000 px the Exercise page shows 4 columns: `main` is 968 px wide inside its padding, and four 220 px columns plus three 16 px gaps (928 px) fit, while five (1164 px) do not. (Core concept 6)
- **Step 6.** In device mode (pixel ratio 2), `currentSrc` ends in `photo-one-800.jpg` at 390 and 800 px, and `photo-one-1600.jpg` at 1200 px, as in the right column of the table in Concept 7. If you get a bigger file at a narrow width, press F5 and ask again. (Core concept 7)
- **Step 7.** The answer is a list of widths: one value at 390 px, three at 800 px, four at 1200 px. (Core concept 6)

© 2026 559 Solutions LLC

- **Step 8a.** Without the viewport line, the page shrinks, the text is tiny, and `innerWidth` no longer says 390; the lesson measured 980. (Core concept 3)
- **Step 8b.** With the `@media` block at the top, the header is stacked at 1200 px. Both header rules have the same specificity, and the base rule now comes later, so it wins. (Core concept 4)

Self-assessment questions

1. **Why the span broke the phone layout.** At 390 px, the grid has room for only one 220 px column. Outside the media query, the `.featured` rule works at every width, so the featured card asks for two columns on a phone too. Grid makes a second, narrow column for it, and every other card squeezes into that column. Inside the media query, the rule works only at 700 px and up. There the grid has three or four columns, so the featured card can span two of them, and the other cards still fit. (Project steps 10, 12 and 15)
2. **A mistake you made.** There is no single answer. A good answer names the mistake, quotes what the page or the Console really showed, for example a red error for one photo file, and says how you found the cause, for example by testing at more than one width.
3. **A sixth interest.** First, save one photo three times in `images`, at 400, 800 and 1200 px wide, named `interest-six-400.jpg`, `interest-six-800.jpg` and `interest-six-1200.jpg`. Write down the real size of the 1200 file. Second, add a new `<li class="card">` to the grid, with an `<h3>`, a `<figure>` and a `<p>`. Third, give its `<img>` a `srcset` with all three files, `sizes="(min-width: 700px) 33vw, 100vw"`, `width` and `height` from the real 1200 file, `loading="lazy"` and real `alt` text, plus a `<figcaption>`. You do not change the CSS. Fourth, save, press F5 and check the page at 390, 800 and 1200 px in device mode. Fifth, check in the Console that the new photo loads at every width with no red errors. Last, publish and run the site test again on the live site. (The brief; Project steps 3, 8, 11, 13, 17, 18 and 19)

Project checks

- **Brief.** `my-site` holds `index.html`, `about.html`, `interests.html`, `image-one.jpg`, `styles.css` and an `images` folder with three files per interest. All three pages have the same four-link nav. `styles.css` starts with the `box-sizing` rule and ends with the one `@media (min-width: 700px)` block. (The brief)
- **Step 6.** At 390 px the site name sits above the links on the Home page. The links stay in one row while they fit and wrap onto a second line when they do not. (Step 6; Core concept 5)
- **Step 12.** One column at 390 px, three at 800 px, four at 1200 px. The sample measured `342px`, `240px` (three times) and `276px` (four times). (Step 12; Core concept 6)
- **Step 13.** At pixel ratio 2 in device mode, a normal card shows the 800 file at all three widths; the featured card shows the 800 file at 390 px and the 1200 file at 800 and 1200 px, because `sizes` says it is about two thirds of the window on wide screens. (Step 13; Core concept 7)
- **Step 14.** With `minmax(300px, 1fr)`, fewer and wider columns fit at 800 and 1200 px. (Step 14; Core concept 6)

© 2026 559 Solutions LLC

- **Step 15.** With `.featured` outside the media query, the phone layout gets two columns, and the second is narrow: the sample measured `220px 129.172px` at 390 px. The second width depends on your card text; what matters is a second, narrow column. (Step 15; Core concepts 4 and 6)
- **Step 16.** Without `height: auto`, the photos keep the `height` attribute as a fixed height and stretch: 308 px wide and 800 px tall on the sample. (Step 16; Core concept 7)
- **Step 17.** With a typo in the 800 entry, the featured photo breaks only where the browser chooses that file: at 390 px with pixel ratio 2, but not with pixel ratio 1. The Console names the missing file. (Step 17; Core concepts 7 and 8)
- **Finishing work.** `git status` says `nothing to commit, working tree clean`, `git log --oneline` shows `Add a responsive Interests page with responsive images` on top, and the live Interests page shows every photo at 390, 800 and 1200 px with no red errors in the Console. (Part 3, steps 6 to 8)

NEXT

Module 4: Forms and Accessibility.
Free lesson: 559solutions.com/code559/module-04-forms-accessibility/

© 2026 559 Solutions LLC

About 559 Solutions

559 isn't just a name. It's where I started: 559 East 37th Street, in the Ida B. Wells Homes on Chicago's South Side. 559 Solutions carries that start into the work: plain language, steady delivery, and solutions people actually use.

The blueprint and the open door

Every 559 Solutions page is drawn like a floor plan: a wall, a window, another wall, and an open door. The plan is how we solve problems, large or small, in any corner of information technology. That covers a nonprofit's tangled spreadsheet and a company's enterprise architecture alike. Every solution moves through the same four steps:

- **Inform:** name the problem, and why it matters.
- **Explain:** show how the solution works, one idea at a time.
- **Illustrate:** draw it, so everyone sees the same plan.
- **Apply:** build it, and put it to work.

In every module, the same four steps sit behind the badges; there the first one is called Introduce. The open door is the point of the plan: a way in, for clients and learners alike.

Code559

Code559 is the teaching side of 559 Solutions. It teaches more than code. It teaches a developer's mindset: predict before you run, read the error, change one thing at a time, test it, save it, and check it live. You work with the real tools developers use every day: VS Code, Git, GitHub, the browser's DevTools and Node. You build working code you can publish, the way a developer does on the job. Think of it as a day in the life of a developer, built into every module.

No jargon. No drama. Just solutions that serve the mission.

— J. R. Wells

559solutions.com

© 2026 559 Solutions LLC